Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота №3

Методи і технології паралельного програмування

Тема: Засоби створення та керування потоками в паралельних мультипоточних програмах. Класичні проблеми та IPC.

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Лесик В. О.

Київ 2026

ЗМІСТ

1 Мета роботи 3
2 Завдання до виконання 4
3 Обґрунтування обрання фреймворку 5
4 Короткий опис рішень та критичні моменти 6
4.1 Задача 1. Переказ коштів між рахунками 6
4.2 Задача 2. Передача даних між процесами 6
5 Контрольні запитання 7
5.1 Поясніть різницю між м’ютексом та семафором. Коли використовувати кожен з них? 7
5.2 Що таке потокобезпечні структури даних? Наведіть приклади. 7
5.3 Поясніть різницю між песимістичною та оптимістичною синхронізацією. 7
5.4 Що таке lock-free та wait-free алгоритми? 7
5.5 Поясніть патерни Producer-Consumer, Pipeline, Map-Reduce, Fork-Join. . 7
5.6 Поясніть можливі відмінності в реалізації патернів відносно різних фреймворків. 7
6 Висновки 8
А Лістинг коду 9
А.1 src/bin/first.rs 9
А.2 src/bin/second.rs 12
А.3 aux_process.py 14

1 МЕТА РОБОТИ

Ознайомитися з механізмами взаємодії та синхронізації потоків, набути практичних навичок реалізації паралельних алгоритмів з координацією роботи між потоками. Демонстрація класичних проблем паралелізму.

2 ЗАВДАННЯ ДО ВИКОНАННЯ

1. Основне завдання: Реалізувати демонстрацію рішення класичних проблем паралельного програмування Deadlock та Race Condition.
2. Задача 1: Реалізувати симуляцію переказу коштів між банківськими рахунками (>100 рахунків, >1000 потоків). Продемонструвати варіанти програми із проблемою та з її вирішенням.
3. Задача 2: Реалізувати передачу даних між двома процесами різного роду за допомогою трьох різних методів обміну повідомленнями.

3 ОБҐРУНТУВАННЯ ОБРАННЯ ФРЕЙМВОРКУ

Для основного процесу обрано мову Rust, для допоміжного - Python. Rust на рівні компілятора забороняє стан гонитви. Для штучної імітації використано сирі вказівники та блоки unsafe. Це демонструє механіку виникнення проблеми при ручному управлінні пам’яттю. Python обрано для реалізації допоміжного процесу через його простоту в обробці потоків вводу/виводу та мережевих сокетів, що дозволяє зосередитися на архітектурі обміну повідомленнями.

4 КОРОТКИЙ ОПИС РІШЕНЬ ТА КРИТИЧНІ МОМЕНТИ

4.1 Задача 1. Переказ коштів між рахунками

Реалізовано 200 рахунків та 1500 потоків. Стан гонитви створено шляхом одночасного доступу до спільного масиву через сирі вказівники без блокувань. Після завершення всіх транзакцій загальна сума коштів у системі суттєво змінюється через перезапис даних. Для вирішення проблеми кожен рахунок обгорнуто у std::sync::Mutex. Взаємне блокування імітується захопленням двох м'ютексів у довільному порядку. Потоки блокують один одного, і програма зависає. Для вирішення взаємного блокування застосовано ієрархію блокувань. Потік завжди першим блокує м'ютекс рахунку з меншим системним індексом. Це унеможливлює циклічне очікування.

```
--- RACE CONDITION ---
Before: 112889
After: 85063
Diff: -27826

--- RESOLVED ---
Before: 112889
After: 112889
Diff: 0

--- DEADLOCK ---
Hanging...
```

4.2 Задача 2. Передача даних між процесами

Реалізовано обмін числовими даними між процесом Rust та скриптом на Python трьома методами: Pipes, Sockets, Shared File. Pipes використовують стандартні канали вводу/виводу ОС. Дані передаються напряму через оперативну пам'ять, що забезпечує найвищу швидкість. Sockets встановлюють з'єднання через TCP на локальному інтерфейсі. Час виконання трохи більший через накладні витрати на встановлення мережевого з'єднання. Shared File використовує файлову систему для збереження та читання стану. Це найповільніший метод через затримки дискової підсистеми.

```
Pipes: 681 -> 1362 (15ms)
Sockets: 681 -> 1362 (15ms)
File: 681 -> 1362 (15ms)
```

5 КОНТРОЛЬНІ ЗАПИТАННЯ

5.1 Поясніть різницю між м’ютексом та семафором. Коли використовувати кожен з них?

М’ютекс блокує ресурс виключно для одного потоку. Семафор дозволяє доступ до ресурсу визначеній кількості потоків одночасно. М’ютекс використовується для захисту змінних від одночасного запису. Семафор потрібен для лімітування доступу до пулу ресурсів.

5.2 Що таке потокобезпечні структури даних? Наведіть приклади.

Це структури, внутрішня реалізація яких унеможливлює стан гонитви при багатопотоковому доступі. Приклади: канали mpsc, структури Arc та RwLock у Rust.

5.3 Поясніть різницю між песимістичною та оптимістичною синхронізацією.

Песимістична блокує ресурс на весь час роботи з ним, припускаючи високу ймовірність конфлікту. Оптимістична виконує зміни локально, а перед записом перевіряє версію даних. Якщо дані змінилися іншим потоком, операція відхиляється та повторюється.

5.4 Що таке lock-free та wait-free алгоритми?

Lock-free гарантує загальний прогрес системи навіть за умови блокування окремих потоків. Wait-free гарантує, що кожен потік виконає свою операцію за обмежену кількість кроків без очікування інших потоків.

5.5 Поясніть патерни Producer-Consumer, Pipeline, Map-Reduce, Fork-Join.

Producer-Consumer розділяє потоки на ті, що створюють дані, і ті, що їх обробляють через спільну чергу. Pipeline вибудовує потоки у ланцюг поетапної обробки даних. Map-Reduce розділяє великий набір даних на частини, паралельно обчислює їх та зливає результати докупи. Fork-Join рекурсивно розбиває одну задачу на менші підзадачі, які виконуються паралельно та об’єднуються у кінцевий результат.

5.6 Поясніть можливі відмінності в реалізації патернів відносно різних фреймворків.

Різні мови мають різні абстракції. У C++ розпаралелювання часто відбувається через директиви компілятора. У Go патерни реалізуються нативно через горутини та канали зв’язку. Rust вимагає явного керування часом життя об’єктів та обгортання їх у розумні вказівники для безпечної передачі між системними потоками.

6 ВИСНОВКИ

У ході лабораторної роботи на практиці відтворено стан гонитви та взаємне блокування. Застосування строгих правил ієрархії блокувань довело свою ефективність уникнення взаємного блокування без суттєвого ускладнення коду. Дослідження передачі даних між процесами показало, що обмін даними через Pipes є найбільш оптимальним рішенням для споріднених процесів на одній машині. Використання файлів для синхронізації є неефективним і створює значне падіння продуктивності порівняно з методами, що працюють в оперативній пам’яті.

Додаток А
Лістинг коду

A.1 src/bin/first.rs

```
struct Prng {
    state: u64,
}

impl Prng {
    fn new(seed: u64) -> Self {
        Self { state: seed }
    }

    fn next_range(&mut self, max: usize) -> usize {
        self.state =
self.state.wrapping_mul(6364136223846793005).wrapping_add(1442695040888963407)
        ((self.state >> 32) as usize) % max
    }
}

fn run_race_condition(num_accounts: usize, num_threads: usize,
transfers_per_thread: usize) {
    let mut prng = Prng::new(42);
    let mut accounts: std::vec::Vec<i32> = (0..num_accounts).map(|_|
(prng.next_range(1000) + 100) as i32).collect();

    let total_before: i32 = accounts.iter().sum();
    std::println!("--- RACE CONDITION ---");
    std::println!("Before: {}", total_before);

    let ptr_addr = accounts.as_mut_ptr() as usize;

    std::thread::scope(|s| {
        for t_id in 0..num_threads {
            s.spawn(move || {
                let mut local_prng = Prng::new((t_id as u64) + 12345);
                let ptr = ptr_addr as *mut i32;

                for _ in 0..transfers_per_thread {
                    let from = local_prng.next_range(num_accounts);
                    let mut to = local_prng.next_range(num_accounts);
                    while from == to { to =
local_prng.next_range(num_accounts); }

                    let amount = (local_prng.next_range(50) + 1) as
i32;
```

```
                    unsafe {
                        let from_ptr = ptr.add(from);
                        let to_ptr = ptr.add(to);

                        let current_from =
std::ptr::read_volatile(from_ptr);
                        if current_from >= amount {
                            std::ptr::write_volatile(from_ptr,
current_from - amount);
                            let current_to =
std::ptr::read_volatile(to_ptr);
                            std::ptr::write_volatile(to_ptr, current_to
+ amount);
                        }
                    }
                }
            });
        }
    });

    let total_after: i32 = accounts.iter().sum();
    std::println!("After: {}", total_after);
    std::println!("Diff: {}\n", total_after - total_before);
}

fn run_resolved(num_accounts: usize, num_threads: usize,
transfers_per_thread: usize) {
    let mut prng = Prng::new(42);
    let accounts: std::vec::Vec<std::sync::Mutex<i32>> =
(0..num_accounts)
        .map(|_| std::sync::Mutex::new((prng.next_range(1000) + 100) as
i32))
        .collect();

    let total_before: i32 = accounts.iter().map(|acc|
*acc.lock().unwrap()).sum();
    std::println!("--- RESOLVED ---");
    std::println!("Before: {}", total_before);

    std::thread::scope(|s| {
        for t_id in 0..num_threads {
            let accounts_ref = &accounts;
            s.spawn(move || {
                let mut local_prng = Prng::new((t_id as u64) + 9999);
                for _ in 0..transfers_per_thread {
                    let from = local_prng.next_range(num_accounts);
                    let mut to = local_prng.next_range(num_accounts);
                    while from == to { to =
```

```
local_prng.next_range(num_accounts); }

                    let amount = (local_prng.next_range(50) + 1) as
i32;

                    let first = std::cmp::min(from, to);
                    let second = std::cmp::max(from, to);

                    let mut lock1 =
accounts_ref[first].lock().unwrap();
                    let mut lock2 =
accounts_ref[second].lock().unwrap();

                    let (from_bal, to_bal) = if from < to {
                        (&mut *lock1, &mut *lock2)
                    } else {
                        (&mut *lock2, &mut *lock1)
                    };

                    if *from_bal >= amount {
                        *from_bal -= amount;
                        *to_bal += amount;
                    }
                }
            });
        }
    });

    let total_after: i32 = accounts.iter().map(|acc|
*acc.lock().unwrap()).sum();
    std::println!("After: {}", total_after);
    std::println!("Diff: {}\n", total_after - total_before);
}

fn run_deadlock(num_accounts: usize, num_threads: usize,
transfers_per_thread: usize) {
    let mut prng = Prng::new(42);
    let accounts: std::vec::Vec<std::sync::Mutex<i32>> =
(0..num_accounts)
        .map(|_| std::sync::Mutex::new((prng.next_range(1000) + 100) as
i32))
        .collect();

    std::println!("--- DEADLOCK ---");
    std::println!("Hanging...");

    std::thread::scope(|s| {
        for t_id in 0..num_threads {
```

```
            let accounts_ref = &accounts;
            s.spawn(move || {
                let mut local_prng = Prng::new((t_id as u64) + 777);
                for _ in 0..transfers_per_thread {
                    let from = local_prng.next_range(num_accounts);
                    let mut to = local_prng.next_range(num_accounts);
                    while from == to { to =
local_prng.next_range(num_accounts); }

                    let _lock_from =
accounts_ref[from].lock().unwrap();
                    std::thread::yield_now();
                    let _lock_to = accounts_ref[to].lock().unwrap();
                }
            });
        }
    });

    std::println!("Done.");
}

fn main() {
    let num_accounts = 200;
    let num_threads = 1500;
    let transfers_per_thread = 1000;

    run_race_condition(num_accounts, num_threads,
transfers_per_thread);
    run_resolved(num_accounts, num_threads, transfers_per_thread);
    run_deadlock(num_accounts, num_threads, transfers_per_thread);
}
```

A.2 src/bin/second.rs

```
fn run_pipes(num: u32) {
    let start = std::time::Instant::now();

    let mut child = std::process::Command::new("python3")
        .arg("aux_process.py")
        .arg("pipe")
        .stdin(std::process::Stdio::piped())
        .stdout(std::process::Stdio::piped())
        .spawn()
        .unwrap();

    let mut stdin = child.stdin.take().unwrap();
    std::io::Write::write_all(&mut stdin, std::format!("{}\n",
num).as_bytes()).unwrap();
    std::mem::drop(stdin);
```

```
    let mut stdout = child.stdout.take().unwrap();
    let mut response = std::string::String::new();
    std::io::Read::read_to_string(&mut stdout, &mut response).unwrap();

    child.wait().unwrap();
    std::println!("Pipes: {} -> {} ({}ms)", num, response.trim(),
start.elapsed().as_millis());
}

fn run_sockets(num: u32) {
    let start = std::time::Instant::now();
    let listener = std::net::TcpListener::bind("127.0.0.1:0").unwrap();
    let port = listener.local_addr().unwrap().port();

    let mut child = std::process::Command::new("python3")
        .arg("aux_process.py")
        .arg("socket")
        .arg(std::format!("{}", port))
        .spawn()
        .unwrap();

    let (mut stream, _) = listener.accept().unwrap();
    std::io::Write::write_all(&mut stream, std::format!("{}\n",
num).as_bytes()).unwrap();

    let mut response = std::string::String::new();
    std::io::Read::read_to_string(&mut stream, &mut response).unwrap();

    child.wait().unwrap();
    std::println!("Sockets: {} -> {} ({}ms)", num, response.trim(),
start.elapsed().as_millis());
}

fn run_shared_file(num: u32) {
    let start = std::time::Instant::now();
    let file_path = "shared_data.txt";

    std::fs::write(file_path, std::format!("{}", num)).unwrap();

    let mut child = std::process::Command::new("python3")
        .arg("aux_process.py")
        .arg("file")
        .arg(file_path)
        .spawn()
        .unwrap();

    child.wait().unwrap();
```

```
    let response = std::fs::read_to_string(file_path).unwrap();
    std::println!("File: {} -> {} ({}ms)", num, response.trim(),
start.elapsed().as_millis());
    std::fs::remove_file(file_path).unwrap();
}

fn main() {
    let num =
(std::time::SystemTime::now().duration_since(std::time::UNIX_EPOCH).unwrap().a
% 1000) as u32;

    run_pipes(num);
    run_sockets(num);
    run_shared_file(num);
}
```

A.3 aux_process.py

```
import sys
import socket

def main():
    mode = sys.argv[1]

    if mode == "pipe":
        data = sys.stdin.read().strip()
        num = int(data)
        print(f"{num * 2}")

    elif mode == "socket":
        port = int(sys.argv[2])
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("127.0.0.1", port))
        data = s.recv(1024).decode().strip()
        num = int(data)
        s.sendall(f"{num * 2}\n".encode())
        s.close()

    elif mode == "file":
        filepath = sys.argv[2]
        with open(filepath, "r") as f:
            data = f.read().strip()
        num = int(data)
        with open(filepath, "w") as f:
            f.write(f"{num * 2}")

if __name__ == "__main__":
    main()
```